

Explaining eBPF Verifier Logs using LLMs

Bruno Gião

Universidade do Minho
Braga, Portugal
pg57868@alunos.uminho.pt

Inês Castro

Universidade do Minho
Braga, Portugal
pg57550@alunos.uminho.pt

Paula Marques

Universidade do Minho
Braga, Portugal
pg50676@alunos.uminho.pt

Abstract

Recent advances in large language models (LLMs) have shown that these models are now capable of performing acceptable and increasingly reliable analysis of eBPF verifier logs, with some models clearly outperforming others in both accuracy and explanatory depth. Building on these observations, we propose the development of an LLM-powered agent designed to assist developers by automatically interpreting and explaining eBPF verifier output, reducing the otherwise considerable effort required to diagnose verification failures. Furthermore, we outline an extension of this agent to not only analyze logs but also generate corrected, verifier-compliant eBPF programs, thus providing a unified, intelligent workflow for both understanding and repairing eBPF kernel-space code.

1 Introduction

The extended Berkeley Packet Filter (eBPF) as been rapidly adopted as the *de facto* method of observability in Linux ecosystems both in academia and in the industry.

The rise of eBPF’s popularity comes largely from its ability to replace traditional observability via kernel modules, removing the need for kernel recompilation or the harder maintenance of said modules across kernel versions.

However, the process of writing eBPF programs is still difficult.[10] Namely using very restricted versions of low level languages such as Rust [11] or C [12][13], the need to understand kernel internals and a strict static verifier that assures the programs don’t interfere with normal kernel execution. This verifier, however, when it detects an error indicates a short description of what went wrong and a traceback of the error in BPF bytecode.[14] This ends up being cryptic and gives little to no actionable advice [1].

As such, we propose a Large Language Model (LLM) specialized in interpreting eBPF verifier logs, explaining them clearly, and suggesting practical solutions or even correct programs. As such we intend to answer the questions:

1. Can LLMs already correctly explain eBPF verifier logs?
2. Can we train or adapt a LLM to explain eBPF verifier logs?
3. Can we have this LLM generate correct eBPF programs based on a passed eBPF verifier log?

2 Background

2.1 What is eBPF?

eBPF (extended Berkeley Packet Filter) is a technology built into the Linux kernel that allows small, sandboxed programs to run safely inside the kernel without requiring modifications to kernel source code or loading kernel modules. It acts like a “virtual machine” inside the kernel, executing programs written in a restricted instruction set [2] [3].

Originally created for packet filtering (networking), eBPF has evolved into a general-purpose, event-driven execution engine, enabling deep observability, security monitoring, performance analysis, and safe kernel extensibility.

2.2 eBPF Pipeline

Behind this powerful abstraction lies a well-defined pipeline that transforms high-level code into verifiable, high-performance programs running inside the Linux kernel. Understanding this pipeline is essential for anyone developing, debugging, or designing tools around eBPF.

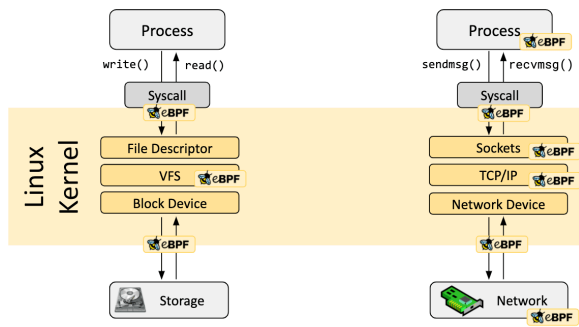


Figure 1: eBPF pipeline [3].

The eBPF lifecycle begins with developers writing their programs in a restricted subset of C or in higher-level DSLs, which are then compiled into eBPF bytecode using Clang/LLVM. The compiler targets a special backend (`-target bpf`) that produces an ELF object file containing the bytecode, BTF type information, and relocation metadata. This packaging step provides the kernel with the structural and semantic details needed to validate and load the program.

The compiled bytecode is passed to the kernel through the `bpf()` syscall, either directly or via libraries such as `libbpf`.

The eBPF verifier is one of the core components of the entire pipeline—and one of the most challenging. Its purpose is to ensure that the program cannot crash the kernel or behave in an unsafe manner. [14] If the program fails verification, the kernel produces diagnostic messages that are often difficult for developers to understand—especially given that these messages reflect deep kernel-level details.

After a program successfully passes verification, it is either interpreted by the kernel's eBPF virtual machine or JIT-compiled into native machine code. [5]

eBPF programs do nothing on their own; they must be attached to a kernel or user-space hook point. When the associated event occurs, the attached eBPF program executes automatically within the kernel context.

To store state or communicate with user-space applications, eBPF relies on maps—kernel-resident data structures such as hash maps, arrays, LRU caches, stacks, queues, and ring buffers. Maps enable multiple eBPF programs—or user space and kernel space—to share data efficiently and safely through well-defined APIs.[15]

2.2.1 The eBPF Verifier

Because eBPF programs execute in the kernel, a faulty program could crash the system or corrupt memory. That's where the verifier comes in, it is a static analysis component in the kernel that inspects an eBPF program, instruction by instruction, before it is allowed to run [4].

It ensures the program is safe, terminating, and follows all kernel rules.

If the program is unsafe, the verifier rejects it and gives an output with the errors.

2.2.1.1 Verification Procedure

The verifier analyzes the program along several dimensions: [14]

- **Memory Safety** - the verifier must establish that all memory accesses are within valid and kernel-allowed bounds.
- **Type and Register State Tracking** - the verifier tracks the abstract type and content of every register at every program instruction. Registers may contain scalars, pointers with specific provenance, or unknown values. Type mismatches, invalid pointer manipulations, and inconsistent register states lead to rejection.
- **Control-Flow Constraints** - the verifier constructs a control-flow graph and checks that all execution paths are finite, branches and loops are statically provable to terminate and all program instructions are reachable and consistent with the tracked state.
- **Stack and Frame Discipline** - eBPF programs are restricted to a maximum 512-byte stack frame. Reading from uninitialized stack memory or storing unsupported types (e.g., untracked pointers) is prohibited.
- **Helper Function Semantics** - only helper functions explicitly exposed by the kernel may be invoked. The verifier checks that the arguments passed to helpers match required types and that the program handles helper return values appropriately (e.g., verifying NULL pointers).

3 Related Work

3.1 Natural-Language–Assisted eBPF and Kernel Development

Recent research has explored the use of large language models (LLMs) to simplify kernel-level programming by enabling developers to specify behavior in natural language rather than low-level C or BPF assembly.

Early efforts such as **Offloading the Tedious Task of Writing eBPF Programs, Kgent: Kernel Extensions Large Language Model Agent** [6], and **KEN: Kernel Extensions Using Natural Language** [7] illustrate the potential of LLMs to translate high-level intent into kernel-side implementations. These systems demonstrate that LLMs can generate boilerplate, manage kernel APIs, and synthesize plausible eBPF or kernel extension code. However, these approaches primarily target code generation from scratch, without addressing the more subtle and critical challenge of making code verifier-compliant.

3.2 LLM-Based Code Understanding and Automated Program Analysis

Beyond the eBPF domain, LLMs have been used to support code comprehension, automated reasoning, and large-scale software analysis. For instance, **Code-Survey: An LLM-Driven Methodology for Analyzing Large-Scale Codebases** [8] demonstrates how LLMs can navigate complex repositories, extract key abstractions, and identify relevant components. This line of work shows that LLMs can effectively reason over structured and semi-structured artifacts—including source code, logs, and documentation—and can provide significant assistance in tasks traditionally requiring deep developer expertise.

3.3 Tools for Interpreting eBPF Verifier Logs

The difficulty of interpreting eBPF verifier output is widely acknowledged within the systems community. As a result, several community-driven projects with repositories [9] have emerged to help developers understand common verifier errors—for example, collections of annotated verifier logs, curated explanations, or scripts that highlight problematic instructions in failing programs.

3.4 Gap in the Literature

Despite advances in natural-language–based kernel programming, LLM-driven code comprehen-

sion, and community efforts to document verifier behavior, no existing system attempts to automatically interpret eBPF verifier error logs and use them to synthesize corrected, verifier-compliant eBPF programs.

To date, no work integrates the abilities we set to implement into a closed-loop system that consumes a verifier error log and code, infers and explains the underlying constraint violation, and produces a corrected version of the eBPF code.

4 A Practical Evaluation of Publicly Available LLMs

To understand the current capabilities and limitations of Large Language Models (LLMs) in assisting with eBPF development, we conducted a practical evaluation using a set of representative models publicly available at the time of writing. The goal of this analysis was to assess how effectively these models could interpret eBPF verifier errors, explain their root causes, and propose correct and actionable fixes.

4.1 LLM Selection

We selected a diverse set of LLMs covering different providers, sizes, and architectural families. The models include both open-source and proprietary systems, spanning general-purpose chat models and code-oriented variants.

- ChatGPT-5;
- Google Gemini-2.5-flash;
- Google Gemini-2.5 Pro;
- Claude-Sonnet-4.5;
- DeepSeek-v3.2;
- Copilot-Resposta Rápida;
- Copilot-Think Deeper;
- Qwen-3 Coder.

We focused our research with the objective of answering to the following questions:

- **Interpretation accuracy:** Can the model correctly understand the eBPF verifier error message?
- **Explanation quality:** Does the model provide a clear, technically correct explanation accessible to developers?
- **Solution soundness:** Are the proposed fixes valid, safe, and aligned with eBPF verifier constraints?

- **Consistency:** Does the model provide similar quality responses across multiple prompts or re-runs?

4.2 Experimental Setup: Prompts and Test Cases

To ensure a controlled and repeatable evaluation, we constructed a set of structured prompts that simulate a realistic developer workflow, such as:

1. “Could you tell me what is the error(s) the eBPF verifier is signaling?”
2. “Could you tell me what the eBPF verifier error is indicating in this format please:”
Cause Description:
Cause Code:
Verifier Error Log:
Solution Description:
Solution Code:
Source:
Kernel Version:
Clang Version:
Additional Remarks:
3. “Using this an example would you change your reply? (this is another program only the structure is the example) <https://github.com/partimenerd/eBPF-verifier-errors/issues/11>”

To begin our evaluation, we submitted the first prompt to all selected LLMs to assess their ability to correctly identify the errors reported by the eBPF verifier. This initial step allowed us to filter out the models that failed to recognize the core issues or produced explanations unrelated to the verifier output.

After identifying the best-performing models, we proceeded with the second prompt, requesting that each LLM present its explanations and proposed solutions in a specific structured format. This format was inspired by a GitHub repository we found that documents common eBPF verifier errors along with clear explanations and fixes. Aligning responses to this structure enabled us to compare the models more consistently and evaluate how well they could adapt to domain-specific formatting requirements.

Finally, we issued a third prompt to validate the technical correctness and ordering of the information provided, ensuring it matched the reference style from the GitHub source and maintained coherence with our own eBPF test programs. This step helped us distinguish between models that merely produced plausible output and those capa-

ble of generating accurate, well-structured, and reproducible guidance.

We designed five representative eBPF programs containing common pitfalls known to challenge the verifier, two of which :

• Program A — Invalid write

We expected the LLM to identify that the write operation targeted memory that is not writable from eBPF, either because it belongs to kernel data structures. We also expected the LLM’s to show us a solution where they would simply remove the instruction.

```
SEC("syscall")
s32 BPF_PROG(tst) {
    struct task_struct *ts =(struct
task_struct*)bpf_get_current_task_btf();
    ts->pid = 0;
    return 0;
}
```

• Program B — Out of Bounds

We expected error of the program attempts to be invalid access to a memory region. It should explain that the eBPF verifier strictly prohibits reads or writes to inaccessible memory regions. will cause the program to fail verification, as such accesses could compromise kernel safety. The solution we expected was to either remove the invalid memory access or ensure that the referenced region is demonstrably accessible.

```
SEC("syscall")
s32 BPF_PROG(tst) {
    struct task_struct *ts = (struct
task_struct*)bpf_get_current_task();
    struct task_struct *p = ts + 1;
    bpf_printk("%d\n", p->pid);
    return 0;
}
```

For each program, we documented the expected explanation and expected solution, which served as the reference baseline.

4.3 Results of our Pratical Evaluation

As described previously, the five program logs were evaluated—both with and without their corresponding code—using the LLMs presented in Section 4.1 and the prompts defined in the preceding section. After comparing the model outputs with our expected solutions, we observed that most LLMs either failed to correctly identify the underlying verifier error or produced inaccurate or incomplete explanations. By excluding the mod-

els that did not meet a satisfactory performance threshold, we narrowed the candidates down to two models that consistently outperformed the others: Claude and Qwen3. Using the correct outputs produced by these models, we proceeded to evaluate the second and third prompts, transforming the responses into the structured format required by our agent.

After obtaining the best possible answers for each test program, we selected the LLM that would serve as the foundation for our agent. Qwen3 was chosen, not only due to its strong performance but also because it is available under a free trial, making it a practical choice for continued experimentation and integration.

5 Architecture

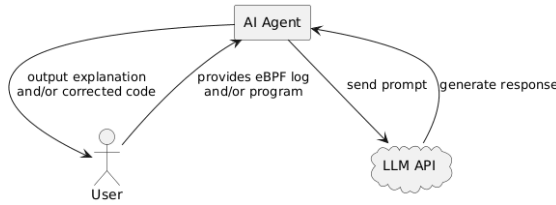


Figure 2: eBPF AI Agent Use Case

The eBPF AI Agent follows a modular architecture designed to analyze verifier errors and automatically generate corrected eBPF programs using a large language model (LLM). The system is composed of three main modules: **Main**, **analyze_ebpf_error**, **solved_file** and an external **LLM API** client.

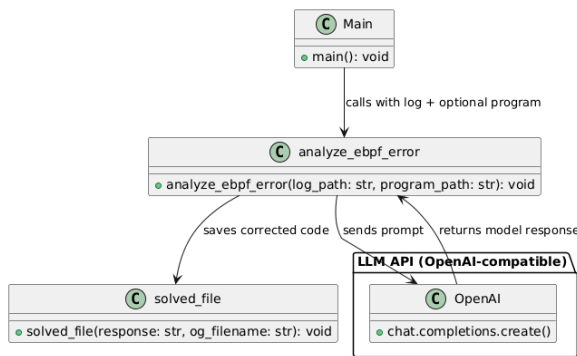


Figure 3: eBPF AI Agent Class Diagram

The **Main** module acts as the entry point, parsing command-line arguments and determining whether the user provides only the verifier log or also the original eBPF program. It then delegates the execution to **analyze_ebpf_error**.

The **analyze_ebpf_error** function is responsible for the core workflow. It loads and validates the input files, constructs a structured prompt containing the verifier log and the original program, when available, and sends it to the LLM model through an Open-AI compatible API. The model returns both a detailed explanation of the verifier error and a proposed corrected version of the program.

The resulting LLM response is then processed by the **solved_file** module, which extracts the corrected code, cleans formatting artifacts, validates its completeness and saves it to a new output file. This module also stores the full model response for our evaluation of the tool.

The modularity ensures maintainability, extensibility and ease of integration with alternative language models or options such as adding which libraries you want the model to utilize.

6 Implementation

We built a prototype as an AI Agent in Python using 168 lines of code, using the libraries `sys`, `os`, `re`, and `OpenAI`. The model chosen for testing was Qwen3 Coder Plus.

The agent can be called on the command line and has as parameters the verifier log directory and, optionally, the origin code location. Then, the agent responds in the terminal filling the following categories: Cause Description, Cause Code, Verifier Error Log, Solution Description, Solution Code, Source, Kernel Version, Clang Version and Additional Remarks. This format was kept consistent with *The eBPF Verifier Errors Project*. To test our results, we made a python script to gather all publicly available submissions to the project and store them in a MongoDB database, where we kept the format consistent as well.

7 Evaluation

7.1 Methodology

To evaluate this solution, we run our prototype on each sample error log in two phases: first with access to the source code, and then using only the verifier log. In addition to the original sample logs, we also include the examples collected from GitHub. Because these logs originate from a variety of libraries, they enable us to assess performance on non-libbpf CO-RE inputs as well.

We test the agent using a modified version of the second prompt from §4, with a small adjustment ensuring that a complete solution program is

always produced. We then evaluate the agent using the same metrics applied to the baseline models.

7.2 Results

Overall, the results are satisfactory and generally improve when source code is provided. This trend is not universal, as sometimes providing source code results in poorer outputs, although this is not significant. It is likely that this phenomenon, in the case of crawled programs, is mainly codedue to samples being incomplete excerpts that do not strictly respect syntax rules, which may confuse the model rather than assist it.

Programs	Correct Problem Description	Correct Solution Description	Generated correct text	Source Code affected output
crawled #1	Y	Y	Y	No
crawled #2	Y	Y	Y	No
crawled #3	Y	Y	N	No
crawled #4	Y	Y	N	Positive
crawled #5	Y	Y	N	Positive
crawled #6	Y	Y	Y	Positive
crawled #7	N	N	N	N/A
crawled #8	N	N	N	Positive
crawled #9	Y	Y	Y	Negatively
crawled #10	Y	Y	N	Positive
prog1	Y	Y	Y	Positive
prog2	Y	Y	N	Positive
prog3	Y	Y	Y	Positive
prog4	Y	Y	Y	Negatively
prog5	Y	Y	N	Positive

Table 1: Evaluation of Agent outputs by program

We also observe that, in the absence of source code, the agent consistently assumes the use of libbpf without CO-RE support, occasionally generating CO-RE specific instructions without refer-

encing vmlinux.h. This leads to inaccuracies in some of the code outputs.

The correctness of the generated code remains inconsistent, influenced by whether source code was provided, by the difficulty of the target program, or even by natural variability across runs.

These findings align with the results obtained in §4, which were broadly positive. The models generally produced high-quality explanations and useful suggestions, indicating that the observed issues may be mitigated through improved prompt engineering, or potentially by adopting the model that performed best in that phase, despite it being a paid option.

8 Conclusions

We successfully performed an analysis of the current capabilities of LLMs in interpreting both eBPF verifier logs and, to a lesser extent, eBPF programs. Our findings show that, overall, modern models demonstrate promising competence in these tasks, with some models notably outperforming the rest. Building on these insights, we also designed an agent capable of explaining verifier errors and generating corresponding eBPF code tailored to those issues.

Despite these encouraging results, the prototype remains only moderately satisfactory. Future work should focus on refining the prompt-engineering strategy to improve both the quality of generated code and the accuracy of diagnostic explanations, particularly for issues that extend beyond the code itself. Additionally, incorporating a user-defined library in the prompt, rather than assuming or inferring one implicitly, would enable more precise and reliable code generation.

9 References

1. “How eBPF Is Powering the Next Generation of Observability” by Nikola Grcevski - <https://thenewstack.io/how-eBPF-is-powering-the-next-generation-of-observability/>
2. “eBPF Overview” - <https://www.datadoghq.com/knowledge-center/eBPF/>
3. “eBPF Documentation” - <https://eBPF.io/what-is-eBPF/>
4. “eBPF Docs” - <https://docs.eBPF.io/linux/concepts/verifier/>

5. “eBPF Architecture” by Hamza Megahed“ - <https://ebpf.hamza-megahed.com/docs/chapter1/5-arch/>
6. “Offloading the Tedious Task of Writing eBPF Programs” - <https://dl.acm.org/doi/abs/10.1145/3748355.3748369>
7. “KEN: Kernel Extensions using Natural Language” - <https://arxiv.org/abs/2312.05531>
8. “Code-Survey: An LLM-Driven Methodology for Analyzing Large-Scale Codebases” - <https://arxiv.org/abs/2410.01837>
9. “The eBPF Verifier Errors Project” by Johannes Bechberger - <https://mostlynerdless.de/blog/2025/01/14/the-ebpf-verifier-errors-project/>
10. “Offloading the Tedious Task of Writing eBPF Programs” (2025) — artigo de pesquisa sobre dificuldades de escrever eBPF. - <https://ratul.org/papers/ebpf2025-simplebpf.pdf>
11. Aya - <https://aya-rs.dev/>
12. eBPF multiple libraries- <https://docs.ebpf.io/ebpf-library/>
13. BCC repository - <https://github.com/iovisor/bcc>
14. “Rex: Closing the language-verifier gap with safe and usable kernel extensions” (2025) - <https://www.usenix.org/system/files/atc25-jia.pdf>
15. BPF maps - <https://docs.kernel.org/bpf/maps.html>